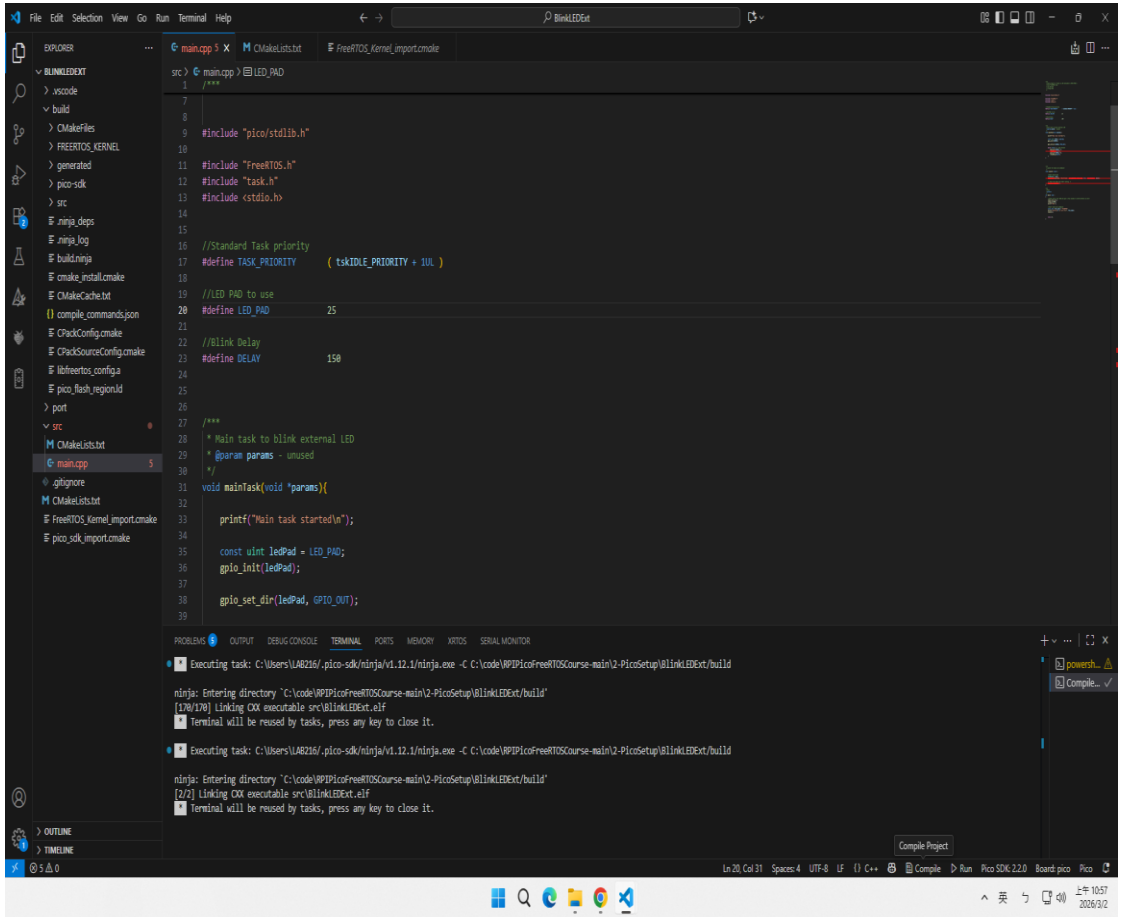
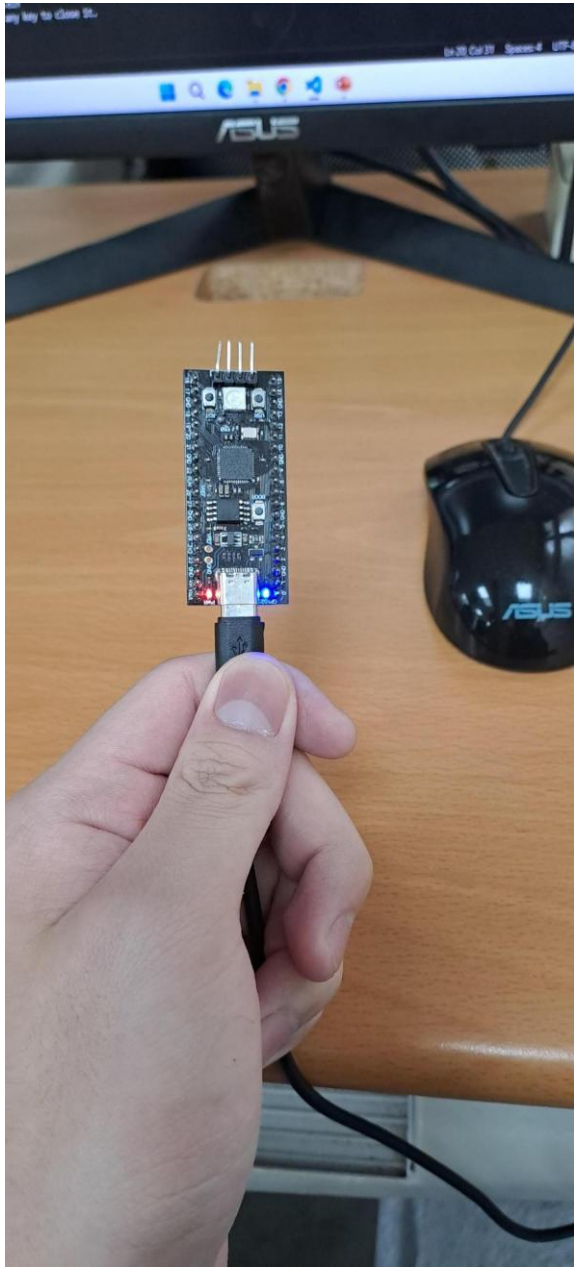
U13157006

子二甲 李恩頡

CH2


File Edit Selection View Go Run Terminal Help
BlinkLEDExt
EXPLORER
BLINKLEDEXT
.vscode
build
CMakeFiles
FREERTOS_KERNEL
generated
pico-sdk
src
.ninja_deps
.ninja_log
build.ninja
cmake_install.cmake
CMakeCache.txt
compile_commands.json
CPackConfig.cmake
CPackSourceConfig.cmake
libfreertos_config.a
pico_flash_region.ld
port
src
CMakeLists.txt
main.cpp 5
.gitignore
CMakeLists.txt
FreeRTOS_Kernel_import.cmake
pico_sdk_import.cmake
main.cpp 5
CMakeLists.txt
FreeRTOS_Kernel_import.cmake
src > main.cpp > LED_PAD
/***
#include "pico/stdlib.h"
#include "FreeRTOS.h"
#include "task.h"
#include <stdio.h>
//Standard Task priority
#define TASK_PRIORITY (tskIDLE_PRIORITY + 1UL)
//LED PAD to use
#define LED_PAD 25
//Blink Delay
#define DELAY 150
/***
* Main task to blink external LED
* @param params - unused
*/
void mainTask(void *params){
printf("Main task started\n");
const uint ledPad = LED_PAD;
gpio_init(ledPad);
gpio_set_dir(ledPad, GPIO_OUT);
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY XRTOS SERIAL MONITOR
Executing task: C:\Users\LAB216\.pico-sdk/ninja/v1.12.1/ninja.exe -C C:\code\RPIPicoFreeRTOSCourse-main\2-PicoSetup\BlinkLEDExt/build
ninja: Entering directory `C:\code\RPIPicoFreeRTOSCourse-main\2-PicoSetup\BlinkLEDExt/build'
[170/170] Linking CXX executable src\BlinkLEDExt.elf
* Terminal will be reused by tasks, press any key to close it.
Executing task: C:\Users\LAB216\.pico-sdk/ninja/v1.12.1/ninja.exe -C C:\code\RPIPicoFreeRTOSCourse-main\2-PicoSetup\BlinkLEDExt/build
ninja: Entering directory `C:\code\RPIPicoFreeRTOSCourse-main\2-PicoSetup\BlinkLEDExt/build'
[2/2] Linking CXX executable src\BlinkLEDExt.elf
* Terminal will be reused by tasks, press any key to close it.
powersh...
Compile...
OUTLINE
TIMELINE
Compile Project
Ln 20, Col 31 Spaces: 4 UTF-8 LF {} C++ Compile Run Pico SDK: 2.2.0 Board: pico Pico
上午 10:57
2026/3/2


ASUS

CH2 程式說明

- 第一個程式是使用 C 語言撰寫的 FreeRTOS LED 閃爍範例。程式在 main 函式中先初始化 USB 序列埠，接著呼叫 vLaunch() 函式來建立任務並啟動 FreeRTOS 排程器。在 vLaunch() 中，透過 xTaskCreate() 建立一個任務，任務執行的函式為 mainTask()，然後呼叫 vTaskStartScheduler() 開始任務排程。mainTask() 內部會初始化 GPIO 腳位，設定為輸出模式，並在無限迴圈中控制 LED 亮與滅，中間透過 vTaskDelay() 進行延遲。整體結構較為單純，所有功能直接以函式方式實作，適合初學者理解 FreeRTOS 的基本運作流程。

CH4

BlinkAgent.cpp > ...

```cpp
/*
 *      Author: jondurrant
 */
// U13157000
#include "BlinkAgent.h"

#include "stdio.h"

//Blink Delay
#define DELAY           150

/***
 * Constructor
 * @param gp - GPIO Pad number for LED
 */
BlinkAgent::BlinkAgent(uint8_t gp) {
    xLedPad = gp;

}

/***
 * Destructor
 */
BlinkAgent::~BlinkAgent() {
    stop();
```

PROBLEMS 23 OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY XRTOS SERIAL MONITOR

Open an additional monitor

Monitor Mode Serial View Mode Text Port COM3 - USB 序列裝置 (COM3) Baud rate 115200 Line ending None

Start Monitoring

```
HEAP avl: 122464, blocks 1, alloc: 16, free: 6
Number of tasks 4
Task: 1          cPri:1          bPri:1          hw:339          MainThread
Task: 2          cPri:0          bPri:0          hw:233          IDLE
Task: 4          cPri:1          bPri:1          hw:87  Blink
Task: 3          cPri:31         bPri:31         hw:970          Tmr Svc
HEAP avl: 122464, blocks 1, alloc: 17, free: 7
Number of tasks 4
```

CH4 程式說明

- 第二個程式則是使用 C++ 撰寫，並採用物件導向的方式設計。程式建立了一個 BlinkAgent 類別，將 LED 控制與 FreeRTOS 任務封裝在類別內。類別包含成員變數用來儲存 LED 腳位與任務控制代碼（TaskHandle），並提供 start()、stop() 等成員函式來管理任務。當呼叫 start() 時，會透過 xTaskCreate() 建立任務，任務入口為靜態函式 vTask()，再由 vTask() 呼叫物件的 run() 函式執行實際工作。在 run() 中，同樣初始化 GPIO 並在無限迴圈中閃爍 LED。此寫法結構清楚，模組化程度較高，未來若需要同時控制多個 LED，只需建立多個 BlinkAgent 物件即可，擴充性與維護性較佳。
- 總結而言，第一個程式屬於傳統 C 語言寫法，結構簡單明確；第二個程式則利用 C++ 物件導向概念進行封裝，使程式更具彈性與可擴充性，也較符合較大型系統開發的設計方式。

兩個程式比較

項目	CH2	CH4
語言	C	C++
結構	單一函式	物件導向
任務建立	直接呼叫	封裝在 class
可擴充性	低	高
多LED管理	較麻煩	可建立多個物件
專業程度	基礎	進階